Optimus Plan — Roadmap Feature Specification

Project: Optimus Plan — Capacity Management Platform **Feature:** Roadmap Management **Version:** 1.0
Date: April 3, 2026

Table of Contents

1. Overview
2. Entity Model
3. Business Use Cases
   - 3.1 Project Management
   - 3.2 Phase Management
   - 3.3 Deliverable Management
   - 3.4 Capacity Demand Analysis
4. Business Rules
5. Architecture Design
   - 5.1 Backend — Hexagonal Architecture
   - 5.2 Database Schema
   - 5.3 REST API
   - 5.4 Frontend
6. Implementation Order
7. Open Questions

---

1. Overview

Purpose

The Roadmap feature extends Optimus Plan with strategic project tracking. It allows managers to define a **Transformation Plan** as a structured hierarchy of phases and deliverables, and — crucially — to answer the question:

> *"Do our teams have enough capacity to deliver this transformation?"*

By linking deliverables to existing teams with estimated man-day effort, the system surfaces a **demand vs. supply** view that bridges high-level planning with the detailed capacity data already computed by Optimus Plan.

Scope

This specification covers three entities — **Project**, **Phase**, and **Deliverable** — and one analytical feature: **Capacity Demand Analysis**. A task-level breakdown (below deliverables) and inter-deliverable dependency tracking are explicitly out of scope for this version.

Relationship to Existing Features

Existing Feature	Integration Point
Teams	Deliverables are assigned to existing teams (soft reference)
Capacity Planning	Capacity demand analysis queries the existing `CapacityCalculator`

Audit Log	All mutations on Project, Phase, and Deliverable are audited
Capacity Snapshots	Future: roadmap demand overlaid on snapshot trend charts

---

2. Entity Model

Structure

```
Project
  └── Phase  (ordered, date-bounded within Project)
        └── Deliverable  (ordered, date-bounded within Phase,
                          optionally assigned to a Team)
```

Project

A **Project** is the top-level container representing a strategic initiative (e.g. "Transformation Plan 2025").

Field	Type	Constraints	Notes
id	UUID	PK	Auto-generated
name	String	NOT NULL, UNIQUE	e.g. "Transformation Plan 2025"
description	String	nullable	Free text
status	Enum	NOT NULL	`DRAFT`, `ACTIVE`, `COMPLETED`, `CANCELLED`
startDate	LocalDate	NOT NULL	
endDate	LocalDate	NOT NULL	Must be after startDate
createdAt	LocalDateTime	NOT NULL	Set on creation
updatedAt	LocalDateTime	NOT NULL	Updated on every change

Status Transitions:

```
DRAFT ──► ACTIVE ──► COMPLETED
  │                     │
  └─────────────────────┴──► CANCELLED
```

No transition is allowed back from `COMPLETED` or `CANCELLED`.

Phase

A **Phase** represents a major time period within a project (e.g. "Phase 1 – Discovery", "Phase 2 – Build").

Field	Type	Constraints	Notes
id	UUID	PK	Auto-generated
projectId	UUID	FK → project, NOT NULL	

name	String	NOT NULL, UNIQUE per project	
displayOrder	int	NOT NULL, UNIQUE per project	Controls timeline ordering
startDate	LocalDate	NOT NULL	Must be within project window
endDate	LocalDate	NOT NULL	Must be within project window, after startDate
status	Enum	NOT NULL	`PLANNED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`

Deliverable

A **Deliverable** is a concrete outcome within a phase. It can be assigned to an existing team with an estimated man-day effort, enabling capacity demand analysis.

Field	Type	Constraints	Notes
id	UUID	PK	Auto-generated
phaseId	UUID	FK → phase, NOT NULL	
name	String	NOT NULL	
description	String	nullable	
status	Enum	NOT NULL	`PLANNED`, `IN_PROGRESS`, `DONE`, `BLOCKED`, `CANCELLED`
priority	Enum	NOT NULL	`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`
assignedTeamId	UUID	nullable	Soft reference to existing Team
estimatedManDays	BigDecimal	nullable, > 0	Used for capacity demand calculations
startDate	LocalDate	NOT NULL	Must be within phase window
endDate	LocalDate	NOT NULL	Must be within phase window, after startDate
displayOrder	int	NOT NULL, UNIQUE per phase	Controls ordering within phase
createdAt	LocalDateTime	NOT NULL	
updatedAt	LocalDateTime	NOT NULL	

> ***Soft Team Reference:*** *`assignedTeamId` is not a hard foreign key with cascade. Deleting a team does not delete deliverables; the UI shows a warning when the referenced team no longer exists.*

---

3. Business Use Cases

3.1 Project Management

Description

A **Project** is created and managed by a Manager. It moves through a defined status lifecycle. Deleting a project cascades to all its phases and deliverables.

Normal Cases

#	Action	Actor	Outcome
1	Create a project	Manager	Project created in `DRAFT` status; audited
2	Activate a project	Manager	Status transitions to `ACTIVE`; requires at least one phase
3	Complete a project	Manager	Status transitions to `COMPLETED`; no new phases can be added
4	Cancel a project	Manager	Status transitions to `CANCELLED`
5	Update project details	Manager	Name, description, or dates updated; audited
6	Delete a project	Manager	Project and all its phases and deliverables removed; audited
7	List all projects	Manager	Full list returned with status, date range, and phase count
8	View full roadmap	Manager	Project returned with all phases and deliverables nested (single query)

Edge Cases

#	Situation	Behaviour
E1	Activate a project with no phases	Rejected — must have at least one phase
E2	Set `endDate` before `startDate`	Rejected
E3	Create a project with a name already in use	Rejected — project names are unique
E4	Update dates such that existing phases fall outside the new window	Rejected — fix or remove the out-of-range phases first
E5	Add a phase to a `COMPLETED` or `CANCELLED` project	Rejected
E6	Transition from `COMPLETED` back to `ACTIVE`	Rejected — terminal state
E7	Delete a project with phases	Cascades — phases and deliverables deleted with a confirmation warning in the UI

3.2 Phase Management

Description

A **Phase** defines a major time segment within a project. Phases are ordered via `displayOrder` and their dates must remain within the project's window.

Normal Cases

#	Action	Actor	Outcome
1	Add a phase to a project	Manager	Phase created with a `displayOrder`; dates validated against project window
2	Reorder phases	Manager	`displayOrder` values updated for the affected phases atomically
3	Update a phase	Manager	Name, dates, or status changed; audited
4	Delete a phase	Manager	Phase and all its deliverables removed; `displayOrder` gap normalised

Edge Cases

#	Situation	Behaviour
E1	Phase dates outside the project window	Rejected
E2	Phase `endDate` before `startDate`	Rejected
E3	Two phases with the same name within the same project	Rejected
E4	Two phases with the same `displayOrder` within the same project	Rejected — unique constraint
E5	Delete a phase that has deliverables	Cascades; UI shows a confirmation warning listing affected deliverables
E6	Phase dates overlap with another phase's dates	Allowed — overlapping phases are valid (parallel workstreams)
E7	Update phase dates such that deliverables fall outside the new window	Rejected — fix or move deliverables first

---

3.3 Deliverable Management

Description

A **Deliverable** represents a concrete outcome within a phase. It can be assigned to a team and given an effort estimate for capacity demand tracking.

Normal Cases

#	Action	Actor	Outcome
1	Add a deliverable to a phase	Manager	Deliverable created; dates validated against phase window; audited

2	Assign a team to a deliverable	Manager	`assignedTeamId` set; deliverable becomes eligible for capacity demand analysis
3	Set estimated man-days	Manager	Stored and used in demand vs. supply calculations
4	Change deliverable status	Manager	Status updated; audited
5	Change deliverable priority	Manager	Priority updated; audited
6	Reorder deliverables within a phase	Manager	`displayOrder` values updated atomically
7	Move a deliverable to another phase	Manager	`phaseId` updated; dates re-validated against the target phase window
8	Update deliverable details	Manager	Any field updated; audited
9	Delete a deliverable	Manager	Removed; audited

Edge Cases

#	Situation	Behaviour
E1	Deliverable dates outside the phase window	Rejected
E2	`estimatedManDays` ≤ 0	Rejected
E3	Assigned team is later deleted from the system	`assignedTeamId` retained; UI shows "Team not found" warning
E4	Move a deliverable whose dates fall outside the target phase window	Rejected — adjust dates before moving
E5	Two deliverables with the same `displayOrder` within the same phase	Rejected — unique constraint
E6	Set status to `DONE` while the deliverable's `endDate` is in the future	Allowed — managers can close deliverables early

---

3.4 Capacity Demand Analysis

Description

The Capacity Demand Analysis is the core analytical feature that connects the roadmap to the capacity data already computed by Optimus Plan. For each month in a project's window, the system compares:

- **Demand**: total estimated man-days for deliverables assigned to a team, spread across the months they cover
- **Supply**: the team's computed available capacity for that month (from the existing `CapacityCalculator`)

Man-days spreading rule: When a deliverable spans N calendar months, its demand is spread evenly: `demandPerMonth = estimatedManDays / N`.

Normal Cases

#	Action	Actor	Outcome
1	View demand vs. supply for a project	Manager	For each month in the project window and each team involved, returns total deliverable demand vs. available capacity
2	Check capacity feasibility for a single deliverable	Manager	Shows whether the assigned team has enough remaining capacity each month the deliverable spans
3	View team load across all projects	Manager	For a given team, aggregates all deliverable demand across all projects per month

Edge Cases

#	Situation	Behaviour
E1	Deliverable has no assigned team or no `estimatedManDays`	Excluded from capacity demand calculations
E2	Team has no capacity data for a month	Available capacity shown as 0; demand still plotted
E3	Deliverable spans only part of a month	That month's demand = `estimatedManDays / N` regardless (no partial-month pro-rating in v1)
E4	Multiple deliverables assigned to the same team overlap in time	Demands are summed for that team per month

---

4. Business Rules

#	Rule
BR-1	**Date containment** — Phase dates must be fully within the project's `startDate–endDate`. Deliverable dates must be fully within the parent phase's date window.
BR-2	**Status lifecycle** — `DRAFT → ACTIVE → COMPLETED | CANCELLED`. No backward transitions from `COMPLETED` or `CANCELLED`.
BR-3	**Activation guard** — A project cannot transition to `ACTIVE` unless it has at least one phase.
BR-4	**Immutable terminal states** — No edits (name, dates, phases) are allowed on a `COMPLETED` or `CANCELLED` project.
BR-5	**Man-days spreading** — Deliverable demand is spread evenly across all calendar months the deliverable covers (`estimatedManDays / numberOfMonths`).
BR-6	**Soft team reference** — Deleting a team does not cascade to deliverables. The `assignedTeamId` is retained as a dangling reference; the UI warns when the team no longer exists.
BR-7	**Audit trail** — All create, update, and delete operations on Project, Phase, and Deliverable produce an entry in the existing `AuditLog`.
BR-8	**Unique phase names per project** — Two phases within the same project cannot share the same name.

BR-9	**Positive estimates** — `estimatedManDays` must be strictly greater than 0 when provided.
BR-10	**Display order uniqueness** — `displayOrder` must be unique per project (for phases) and per phase (for deliverables).

---

5. Architecture Design

5.1 Backend — Hexagonal Architecture

The Roadmap feature follows the same hexagonal (Ports & Adapters) architecture used throughout Optimus Plan.

```
domain/
  model/
    Project.java
    Phase.java
    Deliverable.java
    enums/
      ProjectStatus.java
      PhaseStatus.java
      DeliverableStatus.java
      Priority.java
  port/
    in/
      CreateProjectUseCase.java
      UpdateProjectUseCase.java
      DeleteProjectUseCase.java
      GetProjectUseCase.java
      AddPhaseUseCase.java
      UpdatePhaseUseCase.java
      DeletePhaseUseCase.java
      AddDeliverableUseCase.java
      UpdateDeliverableUseCase.java
      DeleteDeliverableUseCase.java
      GetRoadmapUseCase.java
      GetCapacityDemandUseCase.java
    out/
      ProjectPort.java
      PhasePort.java
      DeliverablePort.java

application/
  roadmap/
    ProjectApplicationService.java
    PhaseApplicationService.java
    DeliverableApplicationService.java
    RoadmapCapacityDemandService.java   ← integrates with existing CapacityCalculator

infrastructure/
  adapter/
```

```
    in/web/
      RoadmapController.java
    out/persistence/
      entity/
        ProjectJpaEntity.java
        PhaseJpaEntity.java
        DeliverableJpaEntity.java
      jpa/
        ProjectJpaRepository.java
        PhaseJpaRepository.java
        DeliverableJpaRepository.java
      adapter/
        ProjectPersistenceAdapter.java
        PhasePersistenceAdapter.java
        DeliverablePersistenceAdapter.java
```

5.2 Database Schema

Three new tables added via a Liquibase migration:

```
CREATE TABLE project (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name         VARCHAR(255) NOT NULL UNIQUE,
    description  TEXT,
    status       VARCHAR(20)  NOT NULL DEFAULT 'DRAFT',
    start_date   DATE         NOT NULL,
    end_date     DATE         NOT NULL,
    created_at   TIMESTAMP    NOT NULL DEFAULT now(),
    updated_at   TIMESTAMP    NOT NULL DEFAULT now(),
    CHECK (end_date > start_date),
    CHECK (status IN ('DRAFT','ACTIVE','COMPLETED','CANCELLED'))
);

CREATE TABLE phase (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id    UUID         NOT NULL REFERENCES project(id) ON DELETE CASCADE,
    name          VARCHAR(255) NOT NULL,
    display_order INT          NOT NULL,
    start_date    DATE         NOT NULL,
    end_date      DATE         NOT NULL,
    status        VARCHAR(20)  NOT NULL DEFAULT 'PLANNED',
    CHECK (end_date > start_date),
    CHECK (status IN ('PLANNED','IN_PROGRESS','COMPLETED','CANCELLED')),
    UNIQUE (project_id, name),
    UNIQUE (project_id, display_order)
);

CREATE TABLE deliverable (
    id                 UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
    phase_id           UUID           NOT NULL REFERENCES phase(id) ON DELETE
CASCADE,
    name               VARCHAR(255)   NOT NULL,
```

```sql
    description        TEXT,
    status             VARCHAR(20)    NOT NULL DEFAULT 'PLANNED',
    priority           VARCHAR(20)    NOT NULL DEFAULT 'MEDIUM',
    assigned_team_id   UUID,          -- soft reference, no FK constraint
    estimated_man_days NUMERIC(10,2)  CHECK (estimated_man_days > 0),
    start_date         DATE           NOT NULL,
    end_date           DATE           NOT NULL,
    display_order      INT            NOT NULL,
    created_at         TIMESTAMP      NOT NULL DEFAULT now(),
    updated_at         TIMESTAMP      NOT NULL DEFAULT now(),
    CHECK (end_date >= start_date),
    CHECK (status   IN ('PLANNED','IN_PROGRESS','DONE','BLOCKED','CANCELLED')),
    CHECK (priority IN ('LOW','MEDIUM','HIGH','CRITICAL')),
    UNIQUE (phase_id, display_order)
);
```

5.3 REST API

Method	Endpoint	Description
POST	/api/projects	Create a project
GET	/api/projects	List all projects
GET	/api/projects/{id}	Get a project (with phases and deliverables)
PUT	/api/projects/{id}	Update a project
PATCH	/api/projects/{id}/status	Transition project status
DELETE	/api/projects/{id}	Delete a project
POST	/api/projects/{id}/phases	Add a phase
PUT	/api/projects/{id}/phases/{phaseId}	Update a phase
DELETE	/api/projects/{id}/phases/{phaseId}	Delete a phase
PATCH	/api/projects/{id}/phases/reorder	Reorder phases
POST	/api/projects/{id}/phases/{phaseId}/deliverables	Add a deliverable
PUT	/api/projects/{id}/phases/{phaseId}/deliverables/{dId}	Update a deliverable
DELETE	/api/projects/{id}/phases/{phaseId}/deliverables/{dId}	Delete a deliverable
PATCH	/api/projects/{id}/phases/{phaseId}/deliverables/reorder	Reorder deliverables
GET	/api/projects/{id}/capacity-demand	Demand vs. supply analysis for the project
GET	/api/teams/{teamId}/roadmap-load	All deliverables assigned to a team across projects

5.4 Frontend

New Angular feature module: `features/roadmap/`

```
features/roadmap/
  roadmap-list/
    roadmap-list.component.ts      — list of all projects with status badges
  roadmap-detail/
    roadmap-detail.component.ts    — project form + phases/deliverables tree
  roadmap-timeline/
    roadmap-timeline.component.ts  — Gantt / timeline view (ngx-gantt or vis-timeline)
  roadmap-capacity/
    roadmap-capacity.component.ts  — demand vs. supply bar chart (reuses ngx-charts)
```

Navigation entry: Added to the existing sidebar shell between "Capacity" and "Snapshots".

---

6. Implementation Order

Step	Scope	Details
1	**Database**	Liquibase migration: 3 new tables (`project`, `phase`, `deliverable`)
2	**Domain layer**	`Project`, `Phase`, `Deliverable` models; status/priority enums; 12 port interfaces
3	**Application services**	`ProjectApplicationService`, `PhaseApplicationService`, `DeliverableApplicationService`
4	**Persistence adapters**	JPA entities, Spring Data repos, persistence adapters
5	**REST controller**	`RoadmapController` with all 16 endpoints
6	**Angular – Project CRUD**	Project list + create/edit form
7	**Angular – Phase & Deliverable CRUD**	Phase/deliverable management within project detail
8	**Angular – Timeline view**	Gantt chart showing phases and deliverables
9	**Capacity demand service**	`RoadmapCapacityDemandService` integrating with existing `CapacityCalculator`
10	**Angular – Capacity demand chart**	Demand vs. supply bar chart per month per team

---

7. Open Questions

The following decisions should be confirmed before implementation begins:

#	Question	Impact

Q1	**Multiple projects?** Should the app support multiple active transformation plans simultaneously, or is there always exactly one active roadmap?	Affects the project list UI and whether a "single active project" constraint is needed
Q2	**Deliverable dependencies?** Do deliverables need "blocks / is blocked by" relationships, or is ordering within a phase sufficient?	Adds a dependency table and DAG validation if yes
Q3	**Role-based access?** Is there a need for project ownership or read-only roles, or does everyone have the same Manager role as today?	Impacts authentication/authorisation layer
Q4	**Man-days spreading method**	Even spread per month is proposed. Should it follow a custom curve (front-loaded, back-loaded, or milestone-based)?
Q5	**Gantt library**	Are you open to adding `ngx-gantt` or `vis-timeline` as a new npm dependency?
Q6	**Audit actor**	Who is the "actor" in audit log entries — a fixed system user, or will user authentication be added?

End of Specification — Optimus Plan Roadmap Feature v1.0